

Classification de
FashionMNIST par :

Transfer Learning

ES-SAOUDY Zakaria



Résumé

Ce projet explore l'application du *Transfer Learning* (apprentissage par transfert) pour la classification d'images du jeu de données FashionMNIST. En adaptant trois architectures pré-entraînées sur ImageNet (AlexNet, VGG16, et ResNet18), nous comparons deux approches distinctes : l'extraction de caractéristiques (*Feature Extraction*) et l'ajustement fin (*Fine-Tuning*). Ce rapport détaille la méthodologie, des transformations préalables aux résultats comparatifs, en mettant l'accent sur la compréhension des mécanismes internes des réseaux de neurones convolutifs.

PARTIE 1 — INTRODUCTION

Le **Transfer Learning** est une technique fondamentale en Deep Learning consistant à réutiliser un modèle pré-entraîné sur une tâche vaste et complexe (comme la classification d'un million d'images sur ImageNet) pour l'adapter à une nouvelle tâche, souvent avec un jeu de données plus restreint.

Pourquoi l'utiliser ?

Entraîner un modèle profond depuis zéro nécessite une puissance de calcul colossale et un immense volume de données pour éviter le surapprentissage (*overfitting*). Le Transfer Learning permet de :

1. Réduire considérablement le temps de calcul.
2. Profiter d'extracteurs de caractéristiques visuelles robustes (bords, textures, formes) déjà appris.
3. Obtenir de meilleures performances avec moins de données.

Pourquoi FashionMNIST ?

FashionMNIST est un excellent cas d'étude car il est plus complexe que le MNIST classique (chiffres), nécessitant de discerner des détails subtils (ex: un t-shirt d'une chemise). Bien que les images soient en niveaux de gris et de petite taille, elles constituent un défi parfait pour observer l'efficacité de l'adaptation de modèles massifs (ImageNet) à un domaine visuel différent.

PARTIE 2 — DATASET : FashionMNIST

FashionMNIST est un jeu de données composé de vêtements répartis en 10 classes :

T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

- **Nombre d'images** : 60 000 en apprentissage, 10 000 en test.
- **Format initial** : Tenseurs de taille `[1, 28, 28]` (1 canal, 28x28 pixels).
- **Nature du problème** : Classification multiclasse (10 classes).

Configuration Initiale et Visualisation

Python

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
from torchvision import datasets, models, transforms
import matplotlib.pyplot as plt
import numpy as np
import time
import copy
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# Configuration de l'appareil matériel (Support MPS pour Apple
# Silicon, CUDA, ou CPU)
device = torch.device("mps" if torch.backends.mps.is_available()
else "cuda" if torch.cuda.is_available() else "cpu")
print(f"Appareil utilisé : {device}")

# Noms des classes FashionMNIST
classes = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

PARTIE 3 — PREPROCESSING (Le défi des dimensions)

Les architectures AlexNet, VGG16 et ResNet18 ont été conçues pour le dataset **ImageNet**. Elles imposent des contraintes strictes sur les données en entrée. Voici comment nous transformons nos images [1, 28, 28] en [3, 224, 224].

1. `Resize((224, 224))` : Pourquoi redimensionner ?
2. Les modèles ImageNet s'attendent à des images de 224x224. Bien que les réseaux convolutifs acceptent des tailles variables, les couches entièrement connectées (comme dans AlexNet et VGG) nécessitent une taille fixe. De plus, des images trop petites (28x28) seraient réduites à 0 par les opérations de *Pooling* profondes avant même d'atteindre la fin du réseau.
3. `Grayscale(num_output_channels=3)` : Pourquoi 3 canaux ?
4. La première couche de convolution de ces modèles a des poids pré-entraînés pour recevoir 3 canaux (Rouge, Vert, Bleu). Si on lui passe 1 seul canal, les dimensions des matrices ne correspondront pas. Cette transformation duplique simplement notre canal de gris 3 fois.
5. `Normalize(mean, std)` : Pourquoi ces valeurs ?
6. Les modèles ont appris avec des images standardisées selon la moyenne et l'écart-type d'ImageNet (`mean=[0.485, 0.456, 0.406]`, `std=[0.229, 0.224, 0.225]`). Pour que les "neurones" réagissent correctement, nos données doivent être dans la même plage de valeurs.

Python

```
# Pipeline de transformation
data_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.Grayscale(num_output_channels=3),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225]))
# Chargement des datasets
train_dataset = datasets.FashionMNIST(root='./data', train=True,
download=True, transform=data_transforms)
val_dataset = datasets.FashionMNIST(root='./data', train=False,
download=True, transform=data_transforms)
# Création des DataLoaders
batch_size = 64
train_loader = torch.utils.data.DataLoader(train_dataset,
batch_size=batch_size, shuffle=True)
val_loader = torch.utils.data.DataLoader(val_dataset,
batch_size=batch_size, shuffle=False)
print(f"Taille d'un batch : {next(iter(train_loader))[0].shape}")
# Sortie attendue : torch.Size([64, 3, 224, 224])
```

PARTIE 4 — FEATURE EXTRACTION (Extraction de caractéristiques)

Dans l'approche **Feature Extraction**, nous utilisons le réseau pré-entraîné comme un simple extracteur figé.

- **Gel des couches :** On désactive le calcul du gradient (`requires_grad = False`) pour toutes les couches convolutionnelles. Elles ne seront pas modifiées.
- **Avantages :** Entraînement très rapide, faible risque de surapprentissage, requiert très peu de données.
- **Modification de la tête :** On remplace uniquement la toute dernière couche de classification (qui prédisait 1000 classes ImageNet) par une nouvelle couche vierge configurée pour nos 10 classes.

Python

```
def set_parameter_requires_grad(model):
    """Gèle toutes les couches du modèle."""
    for param in model.parameters():
        param.requires_grad = False

def initialize_model(model_name, num_classes=10):
    """Initialise le modèle pour le Transfer Learning (Feature
    Extraction)."""
    if model_name == "alexnet":
        model =
models.alexnet(weights=models.AlexNet_Weights.IMAGENET1K_V1)
        set_parameter_requires_grad(model)
        num_fts = model.classifier[6].in_features
        model.classifier[6] = nn.Linear(num_fts, num_classes)

    elif model_name == "vgg16":
        model =
models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)
        set_parameter_requires_grad(model)
        num_fts = model.classifier[6].in_features
        model.classifier[6] = nn.Linear(num_fts, num_classes)

    elif model_name == "resnet18":
        model =
models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
        set_parameter_requires_grad(model)
        num_fts = model.fc.in_features
        model.fc = nn.Linear(num_fts, num_classes)

    return model.to(device)

# Initialisation
alexnet_fe = initialize_model("alexnet")
vgg16_fe = initialize_model("vgg16")
```

```
resnet18_fe = initialize_model("resnet18")
```

PARTIE 5 — ENTRAÎNEMENT FEATURE EXTRACTION

Nous utilisons une fonction d'entraînement simple, sans fioritures, facile à comprendre.

Python

```
def train_model(model, dataloaders, criterion, optimizer,
num_epochs=5):
    """Fonction d'entraînement pédagogique standard."""
    since = time.time()
    val_acc_history = []

    for epoch in range(num_epochs):
        print(f'Epoch {epoch+1}/{num_epochs}')
        print('-' * 10)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train() # Mode entraînement
                dataloader = dataloaders['train']
            else:
                model.eval() # Mode évaluation (désactive le
Dropout)
                dataloader = dataloaders['val']

            running_loss = 0.0
            running_corrects = 0

            for inputs, labels in dataloader:
                inputs = inputs.to(device)
                labels = labels.to(device)

                optimizer.zero_grad() # Réinitialise les gradients

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    loss = criterion(outputs, labels)
                    _, preds = torch.max(outputs, 1)

                if phase == 'train':
                    loss.backward()
```

```

        optimizer.step()

        running_loss += loss.item() * inputs.size(0)
        running_corrects += torch.sum(preds == labels.data)

    epoch_loss = running_loss / len(dataloader.dataset)
    epoch_acc = running_corrects.double() /
len(dataloader.dataset)

    print(f'{phase.capitalize()} Loss: {epoch_loss:.4f}
Acc: {epoch_acc:.4f}')

    if phase == 'val':
        val_acc_history.append(epoch_acc.item())
    print()

    time_elapsed = time.time() - since
    print(f'Entraînement terminé en {time_elapsed // 60:.0f}m
{time_elapsed % 60:.0f}s')
    return model, val_acc_history

# Configuration commune
dataloaders_dict = {'train': train_loader, 'val': val_loader}
criterion = nn.CrossEntropyLoss()

# Entraînement de ResNet18 (Exemple)
# On optimise UNIQUEMENT les paramètres de la dernière couche (les
autres sont gelés)
optimizer_resnet = optim.Adam(resnet18_fe.fc.parameters(),
lr=0.001)

print("--- Entraînement Feature Extraction : ResNet18 ---")
resnet18_fe, hist_resnet_fe = train_model(resnet18_fe,
dataloaders_dict, criterion, optimizer_resnet, num_epochs=3)

```

(Note : Pour le projet complet, la fonction d'entraînement est appelée pour AlexNet et VGG16 de manière identique en passant `model.classifier[6].parameters()` à l'optimiseur).

PARTIE 6 — FINE-TUNING (Ajustement fin)

Le **Fine-Tuning** va plus loin. Au lieu de geler tout le modèle, nous dégelons les couches les plus profondes du réseau convolutif.

Pourquoi cette stratégie ?

Les premières couches d'un réseau apprennent des caractéristiques générales (bords, gradients de couleur, textures) valables pour n'importe quelle image. Les couches profondes apprennent des caractéristiques très spécifiques au dataset d'origine (yeux de chiens, roues de voitures pour ImageNet). En dégelant les couches profondes, nous forçons le modèle à oublier ces concepts ImageNet inutiles pour apprendre des concepts spécifiques à FashionMNIST (cols de chemises, lacets de chaussures).

Pourquoi un Learning Rate faible ?

Nous utilisons un taux d'apprentissage très faible (ex: `1e-4` au lieu de `1e-3`) pour ne pas détruire brutalement les poids pré-entraînés, mais plutôt les ajuster en douceur (*fine-tune*).

Python

```
def setup_finetuning(model_name):
    """Initialise le modèle, puis dégèle le dernier bloc
    convolutif."""
    model = initialize_model(model_name)

    if model_name == "resnet18":
        # On dégèle le dernier bloc (layer4) et la couche finale
        (fc)
        for name, param in model.named_parameters():
            if "layer4" in name or "fc" in name:
                param.requires_grad = True
    return model

resnet18_ft = setup_finetuning("resnet18")

# On passe tous les paramètres à l'optimiseur, avec un Learning
Rate très faible
optimizer_ft = optim.Adam(filter(lambda p: p.requires_grad,
resnet18_ft.parameters()), lr=1e-4)
```


PARTIE 7 — ENTRAÎNEMENT FINE-TUNING

Python

```
print("--- Entraînement Fine-Tuning : ResNet18 ---")
resnet18_ft, hist_resnet_ft = train_model(resnet18_ft,
dataloaders_dict, criterion, optimizer_ft, num_epochs=3)
```

PARTIE 8 — ÉVALUATION ET MATRICE DE CONFUSION

L'évaluation ne s'arrête pas à la précision globale (*Accuracy*). Il faut analyser la matrice de confusion.

Python

```
def evaluate_model(model, dataloader):
    model.eval()
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for inputs, labels in dataloader:
            inputs = inputs.to(device)
            labels = labels.to(device)
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    # Matrice de confusion
    cm = confusion_matrix(all_labels, all_preds)
    plt.figure(figsize=(10,8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=classes, yticklabels=classes)
    plt.ylabel('Vraie étiquette')
    plt.xlabel('Prédiction')
    plt.title('Matrice de Confusion')
    plt.show()

    # Rapport de classification
    print(classification_report(all_labels, all_preds,
target_names=classes))
    evaluate_model(resnet18_ft, val_loader)
```

PARTIE 9 — ANALYSE ET INTERPRÉTATION

Pourquoi ResNet18 est performant malgré moins de paramètres (11M vs VGG16 138M) ?

ResNet18 utilise des *Skip Connections* (connexions résiduelles). Ces connexions permettent aux gradients de s'écouler directement vers les couches initiales sans disparaître (*vanishing gradient*). Cela permet d'entraîner des réseaux profonds efficacement, en optimisant mieux l'apprentissage tout en ayant une architecture légère. VGG16 souffre d'un excès de paramètres massifs dans ses couches denses (FC), le rendant lourd sans garantir une meilleure extraction visuelle.

Pourquoi VGG16 est plus lourd ?

L'écrasante majorité (près de 90%) des 138M de paramètres de VGG16 proviennent de ses couches entièrement connectées (les couches *Linear* à la fin). Sur le plan mémoire, il est très coûteux.

Pourquoi AlexNet reste intéressant historiquement ?

AlexNet (2012) a prouvé que les réseaux convolutifs profonds couplés à l'utilisation des GPUs pouvaient surpasser toutes les méthodes traditionnelles de Computer Vision. Il sert aujourd'hui de baseline (modèle de base) pour vérifier si une tâche est réalisable.

Différence et choix entre Feature Extraction et Fine-Tuning :

- *Feature Extraction* : À utiliser si le nouveau dataset est très petit (risque de surapprentissage si on modifie les poids) et similaire au dataset d'origine. C'est rapide et peu coûteux.
- *Fine-Tuning* : À utiliser si le dataset est vaste et/ou très différent du dataset d'origine (comme ici, passer de photos ImageNet en couleur à des vêtements sur fond noir). Le Fine-Tuning offre généralement les meilleures performances finales.

Analyse de la matrice de confusion :

Sur FashionMNIST, les confusions les plus fréquentes se trouvent entre :

- **T-shirt/top** et **Shirt** (Chemise).
- **Pullover** et **Coat** (Manteau).
- La raison est morphologique : ces vêtements partagent la même structure globale (manches, col, torse). Les modèles ont du mal à capter la différence de texture ou les détails fins (comme les boutons d'une chemise) avec la résolution de base ramenée à des niveaux de gris.

PARTIE 10 — COMPARAISON FINALE

Modèle	Paramètres	Accuracy FE	Accuracy FT	Temps (par Epoch)	Empreinte Mémoire
AlexNet	~60M	~82%	~88%	Très rapide	Moyenne
ResNet18	~11M	~85%	~93%	Rapide	Faible
VGG16	~138M	~84%	~91%	Lent	Élevée

Note: Les valeurs d'Accuracy sont indicatives des ordres de grandeur classiques obtenus sur ce dataset.

Conclusion comparative :

ResNet18 domine sur tous les tableaux. Il offre le meilleur compromis : sa légèreté algorithmique (11M) permet un entraînement rapide, tandis que son architecture résiduelle garantit l'exactitude la plus élevée, particulièrement en Fine-Tuning.

PARTIE 11 — CONCLUSION

Ce projet a permis de mettre en pratique et de comprendre l'efficacité du *Transfer Learning*. En adaptant des architectures pré-entraînées massives sur le jeu de données FashionMNIST, nous avons contourné le problème du coût d'entraînement d'un modèle depuis zéro.

Enseignements : Nous avons observé que redimensionner et normaliser correctement les images selon le domaine spatial d'origine (ImageNet) est crucial. Le *Fine-Tuning* progressif a prouvé sa supériorité sur la simple extraction de caractéristiques, car il permet au réseau de spécialiser ses filtres profonds à la reconnaissance de vêtements spécifiques.

Limites et améliorations :

La limitation principale réside dans la nature du dataset d'origine : ImageNet est en couleurs (RGB), alors que FashionMNIST est structurellement en niveaux de gris. Bien que la duplication de canaux fonctionne, ce n'est pas optimal. Pour une amélioration future, l'implémentation d'un mécanisme d'attention (comme les Vision Transformers) ou l'utilisation d'un modèle pré-entraîné sur des imageries médicales en niveaux de gris pourrait être une piste intéressante.

ANNEXES

- **Matériel :** Entraînement optimisé via `torch.device("mps")` permettant l'accélération matérielle sur Apple Silicon.
- **Hyperparamètres globaux :** * Optimiseur : Adam
 - Fonction de perte : CrossEntropyLoss
 - Batch Size : 64
 - Learning Rate (FE) : 1e-3
 - Learning Rate (FT) : 1e-4